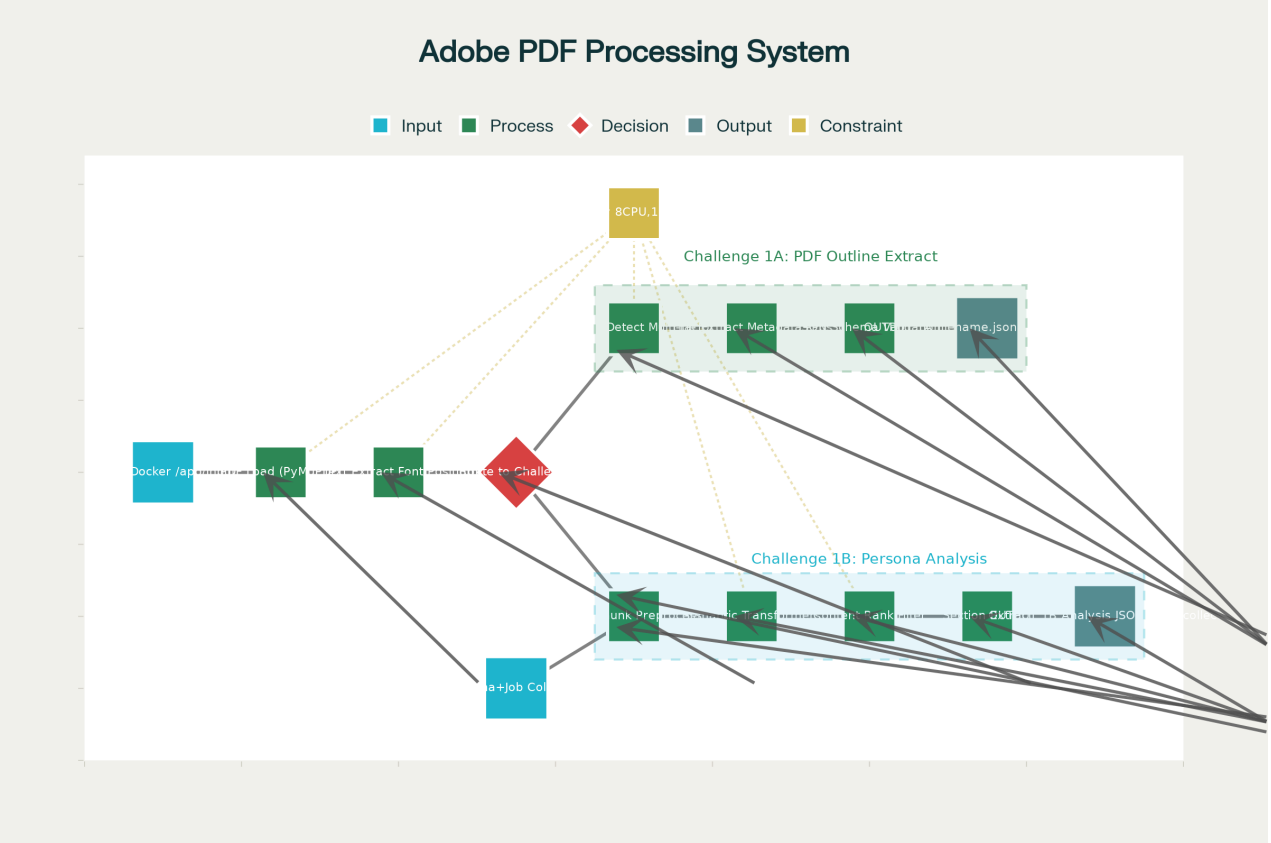
perplexity

Complete Project Flow Guide: Adobe Hackathon PDF Processing System

Project Overview

You're building a sophisticated PDF processing system for the Adobe India Hackathon 2025 "Connecting the Dots" challenge. The system has two main components: **Challenge 1a** (PDF outline extraction) and **Challenge 1b** (persona-driven content analysis). Think of it as creating an intelligent PDF reader that can understand document structure and extract relevant information based on specific user needs.

Complete System Flow: From Input to Output



Complete End-to-End PDF Processing System Flow for Adobe Hackathon Challenges 1A and 1B

The entire system works like an assembly line where PDFs go through multiple processing stages to produce structured JSON outputs. Here's how data flows through the system:

Stage 1: Input Processing

- **What happens**: Your Docker container receives PDF files from the `/app/input` directory
- **Requirements**: Files must be readable, system runs offline (no internet access)
- **Constraints**: CPU-only processing, 8 cores, 16GB RAM available

Stage 2: Core PDF Processing

- **PyMuPDF Library**: Extracts raw text, font information, and positioning data from PDFs
- **Text Analysis**: Identifies font sizes, styles (bold/italic), and spatial relationships
- **Performance**: Single-pass processing for efficiency (extract all data in one go)

Stage 3: Challenge Routing

The system splits into two different processing pipelines:

Challenge 1a: Document Outline Extraction

- **Goal**: Create a table of contents from any PDF
- **Process**: Analyze font sizes and styles to identify headings (H1, H2, H3)
- **Output**: JSON file with document title and hierarchical outline
- **Time Limit**: ≤10 seconds for a 50-page PDF

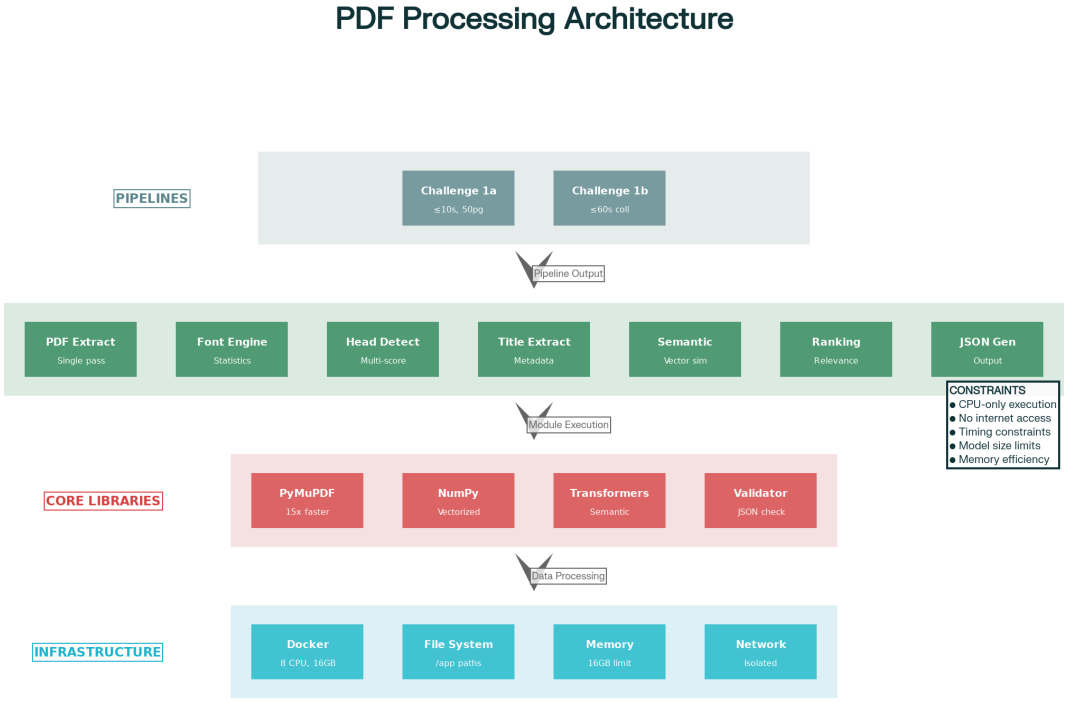
Challenge 1b: Persona-Driven Analysis

- **Goal**: Extract relevant content based on user persona and specific tasks
- **Process**: Use AI to match document sections to user needs
- **Output**: Ranked and filtered content relevant to the persona
- **Time Limit**: ≤60 seconds for document collections

Stage 4: Output Generation

- **JSON Schema Validation**: Ensures outputs match required format exactly
- **File Writing**: Saves results to `/app/output` directory
- **Quality Control**: Validates data integrity and completeness

Technical Architecture Deep Dive



Technical Architecture Layers of the PDF Processing System

The system is built in four distinct layers, each with specific responsibilities:

Layer 1: Infrastructure

- **Docker Container**: Runs on AMD64 architecture, CPU-only
- **File System**: Read-only input, write-only output directories
- **Resource Management**: Strict limits on memory and processing time
- **Network Isolation**: No internet access during execution

Layer 2: Core Libraries

- **PyMuPDF**: The fastest PDF processing library (15x faster than alternatives)
- **NumPy**: Handles mathematical operations and data arrays
- **Sentence Transformers**: AI model for understanding text meaning
- **JSON Validator**: Ensures output format compliance

Layer 3: Application Modules

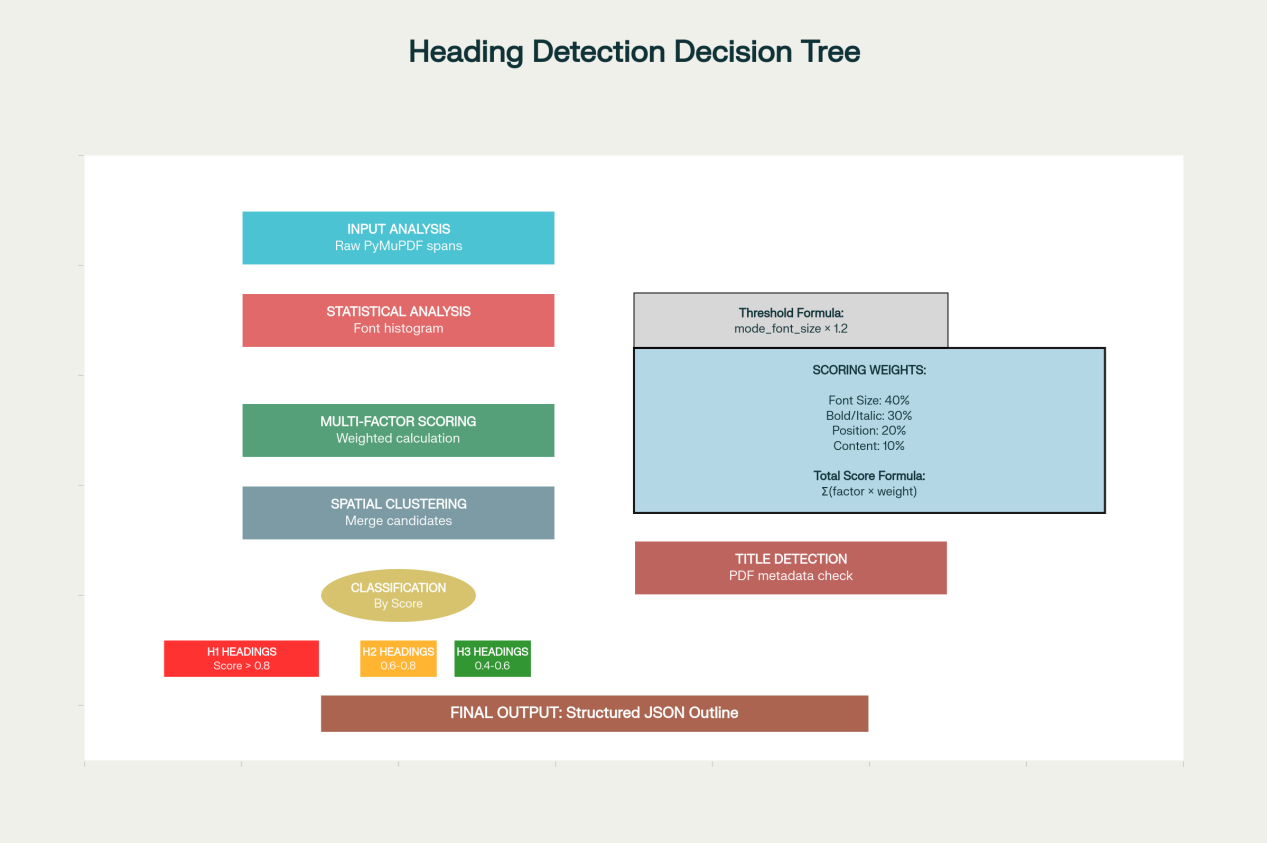
- **PDF Text Extractor**: Pulls text while preserving formatting information
- **Font Analysis Engine**: Identifies different font sizes and styles
- **Heading Detection**: Uses multiple factors to identify document headings
- **Semantic Matching**: AI-powered content relevance scoring

- **JSON Generator**: Creates properly formatted output files

Layer 4: Processing Pipelines

- **Pipeline 1a**: Optimized for speed and outline extraction
- **Pipeline 1b**: Optimized for content understanding and filtering

Challenge 1a: Heading Detection Algorithm Explained



Heading Detection Algorithm Flow with Multi-Factor Scoring System

Challenge 1a is like teaching a computer to read a document the way humans do - by recognizing that bigger, bold text usually represents headings. Here's the step-by-step process:

Step 1: Text Analysis

- Extract every piece of text with its font size, style, and position
- Record whether text is bold, italic, or normal
- Note the location of each text element on the page

Step 2: Statistical Analysis

- Create a histogram of all font sizes in the document
- Find the most common font size (this is likely body text)
- Set heading threshold: anything 1.2× bigger than body text could be a heading

Step 3: Multi-Factor Scoring System

Each text element gets scored based on four factors:

- **Font Size Score (40% weight)**: Larger text = higher score
- **Format Score (30% weight)**: Bold and italic text get bonus points
- **Position Score (20% weight)**: Text at top of pages scores higher
- **Content Score (10% weight)**: ALL CAPS text and numbered sections get points

Formula: `Total Score = (Size × 0.4) + (Format × 0.3) + (Position × 0.2) + (Content × 0.1)`

Step 4: Classification

- **H1 Headlines**: Score > 0.8 (biggest, most prominent text)
- **H2 Subheadings**: Score 0.6-0.8 (medium-large text)
- **H3 Sub-subheadings**: Score 0.4-0.6 (smaller but still prominent)

Step 5: Title Detection

- Check PDF metadata first (sometimes title is embedded)
- Look for largest text in top 20% of first page
- Use filename as fallback if nothing else works

Challenge 1b: Persona-Driven Content Analysis

Challenge 1b is like having an intelligent research assistant that reads documents and finds exactly what you need based on your specific role and task.

Input Format

```
{
  "documents": [{"filename": "doc1.pdf", "title": "Document Title"}],
  "persona": {"role": "Travel Planner"},
  "job_to_be_done": {"task": "Plan a 4-day trip for 10 college friends"}
}
```

Processing Steps

1. **Text Chunking**: Break documents into logical sections
2. **Semantic Analysis**: Use AI to understand meaning of each section
3. **Relevance Scoring**: Match content to persona needs using vector similarity
4. **Content Ranking**: Order sections by importance to the specific task
5. **Extraction**: Pull out the most relevant paragraphs and details

Output Format

```
{
  "extracted_sections": [
    {
      "document": "travel_guide.pdf",
      "section_title": "Budget Planning for Groups",
      "importance_rank": 1,
      "page_number": 15
    }
  ]
}
```

Work Division for 3-Person Team

Person 1: System Architect & Infrastructure Expert

Responsibility: Docker, I/O, and system integration

Day 1 Tasks:

- Set up Docker container with Python 3.12-slim base
- Configure input/output directory handling (`/app/input`, `/app/output`)
- Implement PDF loading with PyMuPDF
- Create basic file iteration and error handling

Day 2 Tasks:

- Build fast text extraction pipeline
- Implement single-pass PDF processing for optimal performance
- Create structured data objects for text spans
- Optimize memory usage and processing speed

Day 3 Tasks:

- Develop JSON output pipeline with schema validation
- Implement strict compliance with hackathon output format
- Handle edge cases (empty PDFs, corrupted files)
- Create document title extraction logic

Day 4 Tasks:

- Integration testing with other team members' components
- Performance optimization and profiling
- Final Docker image building and deployment testing
- Documentation and README creation

Person 2: Heading Detection & Algorithm Specialist

Responsibility: Core Challenge 1a algorithm development

Day 1 Tasks:

- Research and analyze sample PDFs from hackathon repository
- Study font size patterns and heading characteristics
- Design multi-factor scoring algorithm architecture
- Plan heading hierarchy classification system

Day 2 Tasks:

- Implement font size histogram analysis
- Build multi-factor scoring system with proper weights
- Create statistical clustering for font analysis
- Develop content pattern recognition (numbering, capitalization)

Day 3 Tasks:

- Implement spatial clustering for wrapped headings
- Build hierarchical classification (H1/H2/H3) logic
- Handle two-column layouts and complex document structures
- Fine-tune scoring weights based on sample outputs

Day 4 Tasks:

- Test algorithm on all provided sample PDFs
- Adjust heuristics for maximum accuracy
- Debug edge cases and unusual document formats
- Performance optimization for sub-10-second requirement

Person 3: AI/Semantic Analysis & Challenge 1b Expert

Responsibility: Persona-driven content analysis system

Day 1 Tasks:

- Study persona and job-to-be-done requirements from samples
- Design semantic matching architecture using Sentence Transformers
- Plan content relevance scoring methodology
- Research offline AI model requirements (size < 1GB)

Day 2 Tasks:

- Implement text chunking and preprocessing pipeline
- Integrate Sentence Transformers for offline semantic analysis

- Build vector similarity matching between persona needs and content
- Ensure all models work without internet access

Day 3 Tasks:

- Develop content ranking and importance scoring algorithm
- Implement section extraction with relevance justification
- Create multi-document collection processing
- Build structured output formatting for Challenge 1b

Day 4 Tasks:

- Integration with main system pipeline
- Test on all sample persona collections
- Optimize AI model performance for 60-second constraint
- Create explainable ranking rationales

Key Technical Requirements

Performance Constraints

- **Challenge 1a**: Maximum 10 seconds for 50-page PDF
- **Challenge 1b**: Maximum 60 seconds for document collections
- **Model Size**: ≤200MB for Challenge 1a, ≤1GB for Challenge 1b
- **CPU Only**: No GPU acceleration available
- **Memory Limit**: 16GB RAM maximum

JSON Schema Compliance

Both challenges must produce JSON outputs matching exact schemas:

Challenge 1a Output:

```
{
  "title": "Document Title",
  "outline": [
    {
      "level": "H1",
      "text": "Chapter 1: Introduction",
      "page": 1
    }
  ]
}
```

Challenge 1b Output:

```
{
  "metadata": {
    "persona": "Travel Planner",
    "job_to_be_done": "Plan group trip"
  },
  "extracted_sections": [
    {
      "document": "guide.pdf",
      "section_title": "Budget Planning",
      "importance_rank": 1,
      "page_number": 15
    }
  ]
}
```

Docker Deployment

```
FROM --platform=linux/amd64 python:3.12-slim
RUN pip install --no-cache-dir pymupdf numpy sentence-transformers
COPY src/ /app
WORKDIR /app
ENTRYPOINT ["python", "main.py"]
```

Success Metrics

- **Accuracy**: High precision/recall on heading detection
- **Speed**: Meet strict timing requirements
- **Robustness**: Handle various PDF formats and layouts
- **Compliance**: Perfect JSON schema adherence
- **Scalability**: Process multiple documents efficiently

This system transforms static PDF documents into intelligent, structured data that can be queried and analyzed programmatically - essentially building the foundation for next-generation document understanding applications.

⁂